llms-txt

The /llms.txt file

The /llms.txt file

A proposal to standardise on using an `/llms.txt` file to provide information to help LLMs use a website at inference time.

AUTHOR
Jeremy Howard

PUBLISHED
September 3, 2024

Background

Large language models increasingly rely on website information, but face a critical limitation: context windows are too small to handle most websites in their entirety. Converting complex HTML pages with navigation, ads, and JavaScript into LLM-friendly plain text is both difficult and imprecise.

While websites serve both human readers and LLMs, the latter benefit from more concise, expert-level information gathered in a single, accessible location. This is particularly important for use cases like development environments, where LLMs need quick access to programming documentation and APIs.

Proposal

We propose adding a `/llms.txt` markdown file to websites to provide LLM-friendly content. This file offers brief background information, guidance, and links to detailed markdown files.

llms.txt markdown is human and LLM readable, but is also in a precise format allowing fixed processing methods (i.e. classical programming techniques such as parsers and regex).

llms.txt logo

We furthermore propose that pages on websites that have information that might be useful for LLMs to read provide a clean markdown version of those pages at the same URL as the original page, but with `.md` appended. (URLs without file names should append `index.html.md` instead.)

The FastHTML project follows these two proposals for its documentation. For instance, here is the FastHTML docs llms.txt. And here is an example of a regular HTML docs page, along with exact same URL but with a .md extension.

This proposal does not include any particular recommendation for how to process the llms.txt file, since it will depend on the application. For example, the FastHTML project opted to automatically expand the llms.txt to two markdown files with the contents of the linked URLs, using an XML-based structure suitable for use in LLMs such as Claude. The two files are: llms-ctx.txt, which does not include the optional URLs, and llms-ctx-full.txt, which does include them. They are created using the `llms_txt2ctx` command line application, and the FastHTML documentation includes information for users about how to use them.

The versatility of llms.txt files means they can serve many purposes - from helping developers find their way around software documentation, to giving businesses a way to outline their structure, or even breaking down complex legislation for stakeholders. They're just as useful for personal websites where they can help answer questions about someone's CV, for e-commerce sites to explain products and policies, or for schools and universities to provide quick access to their course information and resources.

Note that all nbdev projects now create .md versions of all pages by default. All Answer.AI and fast.ai software projects using nbdev have had their docs regenerated with this feature. For an example, see the markdown version of fastcore's docments module.

Format

At the moment the most widely and easily understood format for language models is Markdown. Simply showing where key Markdown files can be found is a great first step. Providing some basic structure helps a language model to find where the information it needs can come from.

The `llms.txt` file is unusual in that it uses Markdown to structure the information rather than a classic structured format such as XML. The reason for this is that we expect many of these files to be read by language models and agents. Having said that, the information in llms.txt follows a specific format and can be read using standard programmatic-based tools.

The llms.txt file spec is for files located in the root path `/llms.txt` of a website (or, optionally, in a subpath). A file following the spec contains the following sections as markdown, in the specific order:

- An optional byte-order mark (BOM)
- An H1 with the name of the project or site. This is the only required section
- A blockquote with a short summary of the project, containing key information necessary for understanding the rest of the file
- Zero or more markdown sections (e.g. paragraphs, lists, etc) of any type except headings, containing more detailed information about the project and how to interpret the provided files
- Zero or more markdown sections delimited by H2 headers, containing "file lists" of URLs where further detail is available
    - Each "file list" is a markdown list, containing a required markdown hyperlink `[name](url)`, then optionally a `:` and notes about the file.

Here is a mock example:

```
# Title

> Optional description goes here

```

```
Optional details go here

## Section name

- [Link title](https://link_url): Optional link details

## Optional

- [Link title](https://link_url)
```

Note that the "Optional" section has a special meaning—if it's included, the URLs provided there can be skipped if a shorter context is needed. Use it for secondary information which can often be skipped.

Existing standards

llms.txt is designed to coexist with current web standards. While sitemaps list all pages for search engines, `llms.txt` offers a curated overview for LLMs. It can complement robots.txt by providing context for allowed content. The file can also reference structured data markup used on the site, helping LLMs understand how to interpret this information in context.

The approach of standardising on a path for the file follows the approach of `/robots.txt` and `/sitemap.xml`. robots.txt and `llms.txt` have different purposes—robots.txt is generally used to let automated tools know what access to a site is considered acceptable, such as for search indexing bots. On the other hand, `llms.txt` information will often be used on demand when a user explicitly requests information about a topic, such as when including a coding library's documentation in a project, or when asking a chat bot with search functionality for information. Our expectation is that `llms.txt` will mainly be useful for *inference*, i.e. at the time a user is seeking assistance, as opposed to for *training*. However, perhaps if `llms.txt` usage becomes widespread, future training runs could take advantage of the information in `llms.txt` files too.

sitemap.xml is a list of all the indexable human-readable information available on a site. This isn't a substitute for `llms.txt` since it:

- Often won't have the LLM-readable versions of pages listed
- Doesn't include URLs to external sites, even though they might be helpful to understand the information
- Will generally cover documents that in aggregate will be too large to fit in an LLM context window, and will include a lot of information that isn't necessary to understand the site.

Example

Here's an example of `llms.txt`, in this case a cut down version of the file used for the FastHTML project (see also the full version):

```
# FastHTML
```

```
> FastHTML is a python library which brings together Starlette, Uvicorn, HTMX, and
        fastcore's `FT` "FastTags" into a library for creating server-rendered
        hypermedia applications.

Important notes:

- Although parts of its API are inspired by FastAPI, it is *not* compatible with
        FastAPI syntax and is not targeted at creating API services
- FastHTML is compatible with JS-native web components and any vanilla JS library, but
        not with React, Vue, or Svelte.

## Docs

- [FastHTML quick start]
        (https://fastht.ml/docs/tutorials/quickstart_for_web_devs.html.md): A brief
        overview of many FastHTML features
- [HTMX reference]
        (https://github.com/bigskysoftware/htmx/blob/master/www/content/reference.md):
        Brief description of all HTMX attributes, CSS classes, headers, events,
        extensions, js lib methods, and config options

## Examples

- [Todo list application]
        (https://github.com/AnswerDotAI/fasthtml/blob/main/examples/adv_app.py):
        Detailed walk-thru of a complete CRUD app in FastHTML showing idiomatic use of
        FastHTML and HTMX patterns.

## Optional

- [Starlette full documentation]
        (https://gist.githubusercontent.com/jph00/809e4a4808d4510be0e3dc9565e9cbd3/raw/9b
        sml.md): A subset of the Starlette documentation useful for FastHTML development.
```

To create effective `llms.txt` files, consider these guidelines:

- Use concise, clear language.
- When linking to resources, include brief, informative descriptions.
- Avoid ambiguous terms or unexplained jargon.
- Run a tool that expands your `llms.txt` file into an LLM context file and test a number of language models to see if they can answer questions about your content.

Directories

Here are a few directories that list the `llms.txt` files available on the web:

- [llmstxt.site]
- [directory.llmstxt.cloud]

Integrations

Various tools and plugins are available to help integrate the llms.txt specification into your workflow:

- `llms_txt2ctx` - CLI and Python module for parsing llms.txt files and generating LLM context
- JavaScript Implementation - Sample JavaScript implementation
- `vitepress-plugin-llms` - VitePress plugin that automatically generates LLM-friendly documentation for the website following the llms.txt specification
- `docusaurus-plugin-llms` - Docusaurus plugin for generating LLM-friendly documentation following the llmtxt.org standard
- Drupal LLM Support - A Drupal Recipe providing full support for the llms.txt proposal on any Drupal 10.3+ site
- `llms-txt-php` - A library for writing and reading llms.txt Markdown files
- `VS Code PagePilot Extension` - PagePilot is a VS Code Chat participant that automatically loads external context (documentation, APIs, README files) to provide enhanced responses.

Next steps

The `llms.txt` specification is open for community input. A GitHub repository hosts this informal overview, allowing for version control and public discussion. A community discord channel is available for sharing implementation experiences and discussing best practices.

Report an issue